



Functions in JavaScript

Learning Objectives

- Show repetition problem and need for functions
- Define a function and call it multiple times
- Explain parameters and arguments simply
- Teach return values vs console.log
- Show declaration and expression forms
- Introduce arrow functions briefly
- Explain global and local scope
- Practice with small real examples





Why Functions are Needed?

The Problem – Repetition in Code

Imagine writing the same code again and again

Example:

```
console.log("Hello");  
console.log("Hello");  
console.log("Hello");
```

This creates:

- Repetitive code
- Harder to maintain
- More chances of mistakes

Why This is a Problem?

- **If you want to change "Hello" → you must change it everywhere**

Code becomes:

- Messy
- Longer
- Difficult to debug
- This is called code duplication

The Solution – Functions

- **A function is a reusable block of code**
- Write it once, use it many times

Makes code:

- Cleaner
- Easier to manage
- More efficient

Your First Function

```
function greet() {  
    console.log("Hello");  
}
```

```
greet();
```

function greet() → defining the function

greet(); → calling the function

Key idea: Define once, use many times

Pop Quiz

Q. Why do we use functions?

A

To make code longer

B

**To reuse code and avoid
repetition**

Pop Quiz

Q. Why do we use functions?

A

To make code longer

B

**To reuse code and avoid
repetition**



Parameters vs Arguments (Core Understanding)

Extending Our Function

```
function greet(name) {  
  console.log("Hello " + name);  
}  
  
greet("John");
```

Now the function takes input

Output: Hello John

We can make greetings more flexible

Parameters vs Arguments

- **Parameter** → placeholder inside the function
 - Example: name
- **Argument** → actual value passed when calling the function
 - Example: "John"

👉 Think of it like:

- **Parameter = empty box** 
- **Argument = value you put inside the box**

Practice & Predict

```
greet("Alice");
```

```
greet("Rahul");
```

```
greet("Sara");
```

What will be the output?

- A. Hello Alice
- B. Hello Rahul
- C. Hello Sara



👉 **Same function, different outputs!**

Pop Quiz

Q. In `greet("John")`, what is "John"?

A

Argument

B

Parameter

Pop Quiz

Q. In `greet("John")`, what is "John"?

A

Argument

B

Parameter



Return Values

console.log vs return

- Both are used inside functions, but they do different things
- **console.log()**
 - Prints output to the console
- **return**
 - Sends a value back from the function

👉 They are NOT the same

Example Using return

```
function add(a, b) {  
    return a + b;  
}  
  
let result = add(2, 3);  
  
console.log(result);  
  
return a + b; → gives back the result  
result stores the value → 5  
  
console.log(result); → prints 5
```



Key Difference

- **console.log()**
 - Just displays output
 - Cannot store the result easily
- **return**
 - Gives value back
 - Can store it in a variable
 - Can reuse it later

👉 ***return = useful output***



Practice Time

Try writing these:

- Multiply two numbers
- Check if a number is even or odd

👉 Predict outputs before running!





Hands-On Demo | Functions in JavaScript

Pop Quiz

Q. What does return do?

A

**Sends value back from the
function**

B

Prints output on screen

Pop Quiz

Q. What does return do?

A

**Sends value back from the
function**

B

Prints output on screen



Function Declaration vs Expression

Two Ways to Create Functions

There are two common ways to write functions in JavaScript:

- Function Declaration
- Function Expression

 ***Both are valid ways to create functions***

Function Declaration

```
function sayHi() {  
    console.log("Hi");  
}
```

- Starts with the keyword function
- Has a name (*sayHi*)
- Can be called anytime in the code

Function Expression

```
const sayHi = function() {  
  console.log("Hi");  
};
```

- Function is stored in a variable
- No name needed inside the function
- Used like a value

Practice Time

Convert between the two:

Declaration → Expression

```
function greet() {  
  console.log("Hello");  
}
```

Expression → Declaration

```
const add = function(a, b) {  
  return a + b;  
};
```

👉 Try writing both versions yourself!



Pop Quiz

Q. Which statement is true?

A

**Only function declarations
are valid**

B

**Both declarations and
expressions are valid**

Pop Quiz

Q. Which statement is true?

A

**Only function declarations
are valid**

B

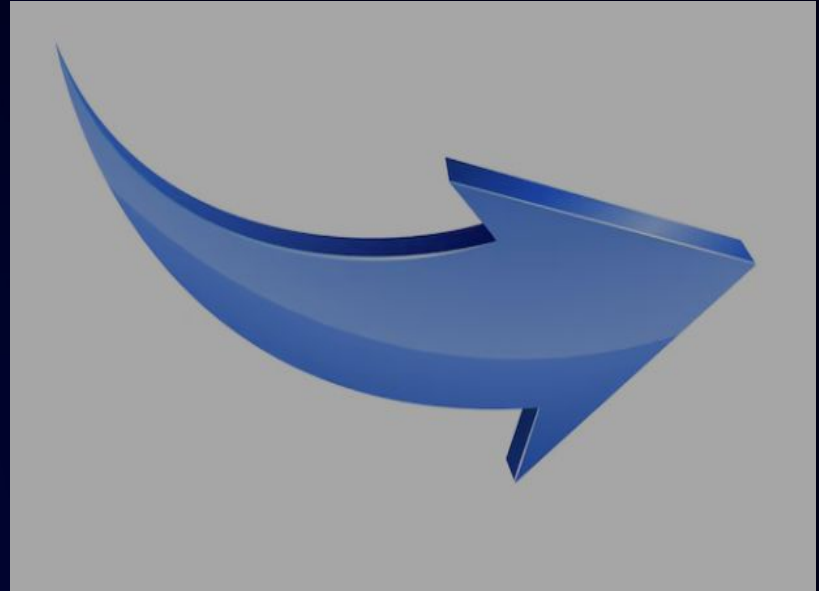
**Both declarations and
expressions are valid**



Arrow Functions

Introduction To Arrow Functions

- Arrow functions are a shorter way to write functions
- Use `=>` symbol
- Introduced in modern JavaScript (ES6)
- Cleaner and more concise syntax
- Often used in callbacks



syntax for writing arrow functions

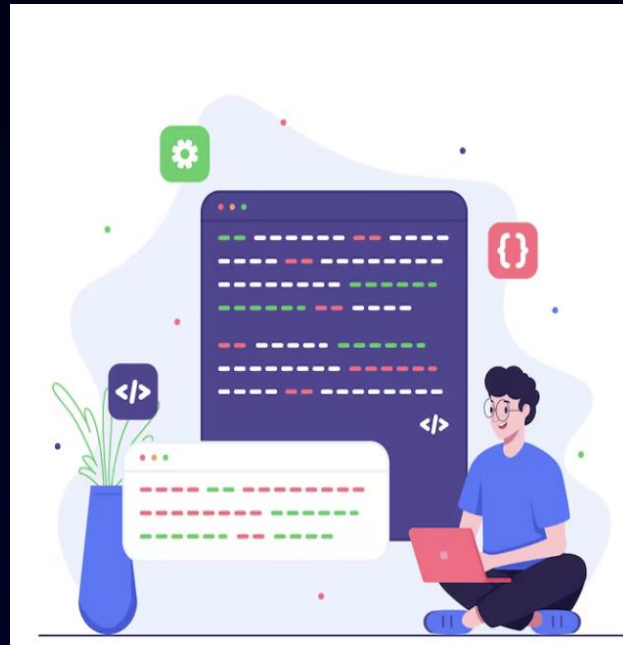
Syntax : `const functionName = (parameters)
=> {
 // code
};`

Examples :

```
const greet = () => "Hello"; //  
No parameter
```

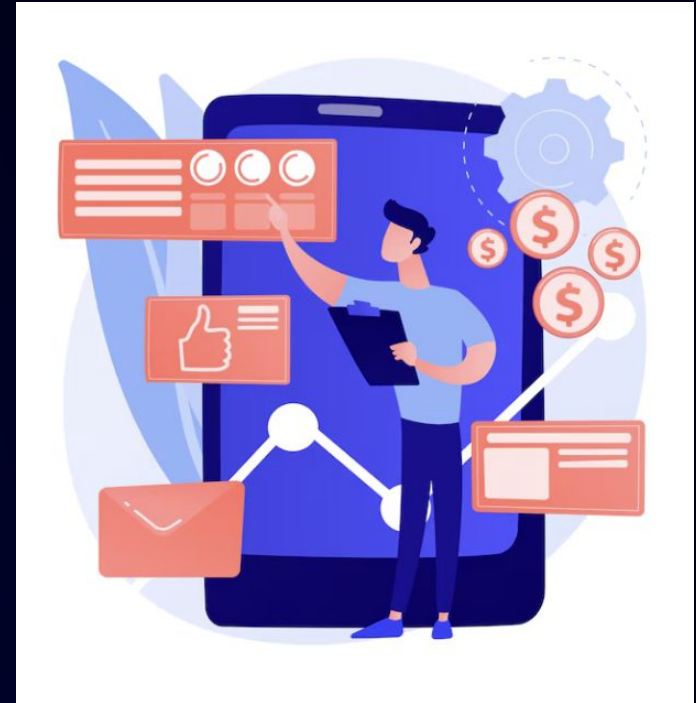
```
const square = x => x * x; //  
One parameter
```

```
const add = (a, b) => a + b; //  
Multiple parameter
```



Widely Used in Modern JavaScript

- Common in modern JavaScript (ES6+)
- Frequently used in callbacks
- Used in array methods like map, filter, forEach
- Helps write shorter and cleaner code
- Preferred in React and modern frameworks
- Makes code more readable
- Used for quick one-line functions





Hands-On Demo | functions using arrow syntax

What is an arrow function?

A

**A function written using =>
syntax**

B

loop used for arrays

What is an arrow function?

A

A function written using =>
syntax

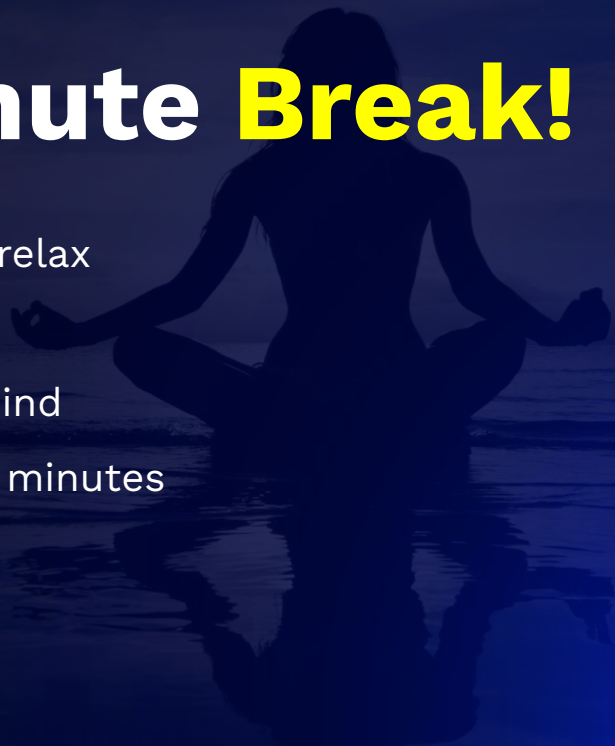
B

loop used for arrays



Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes





Scope (Global vs Local)

Global Vs Local scope :

Global Scope :

- Declared outside any function
- Accessible everywhere in the code
- Can be used by all functions

Local Scope :

- Declared inside a function
- Accessible only within that function
- Not available outside the function



Accessing Local Variable Outside Function

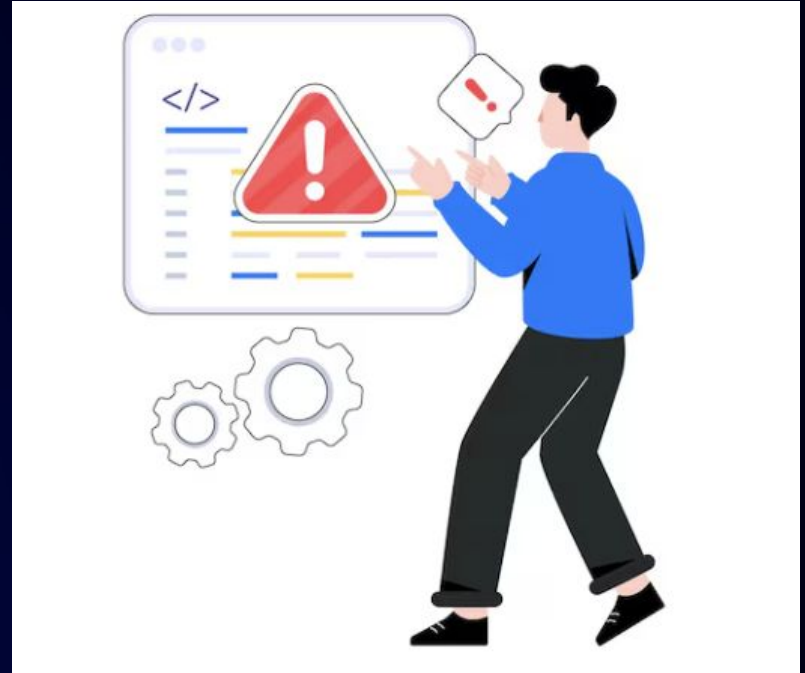
```
function test(){  
  let message = "Hello";  
}  
  
console.log(message);
```

Output:

ReferenceError: message is not
defined

It exists only inside test()

Cannot be accessed outside the
function



Pop Quiz

Which statement is true about local scope?

A

Variable accessible everywhere

B

Variable accessible only inside function

Pop Quiz

Which statement is true about local scope?

A

**Variable accessible
everywhere**

B

**Variable accessible only
inside function**



Pure vs Impure Functions

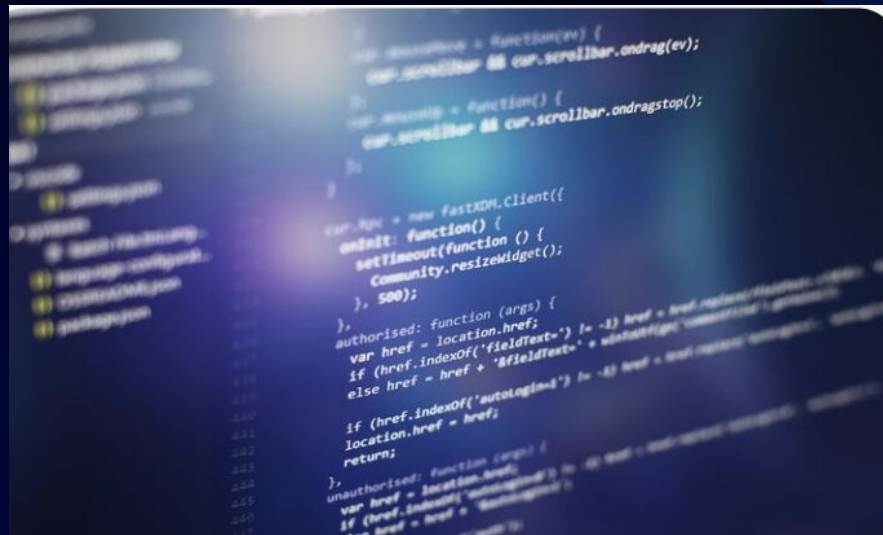
Intro To Pure Function

- Same input gives same output
- No dependency on outside variables
- No side effects
- Predictable behavior
- Easier to test

The word "PURE" is rendered in a bold, sans-serif font. Each letter is filled with a dense, green, grass-like texture, giving it a three-dimensional, organic appearance. The letters are set against a plain white background, which is itself centered within a dark blue rectangular frame.

Intro To Impure Function

- Output can change for same input
- Depends on outside variables
- Has side effects
- Not predictable
- Harder to test



```
var scrollbar = Function(ev) {  
  cur.scrollbar && cur.scrollbar.ondrag(ev);  
};  
  
var scrollbar = Function() {  
  cur.scrollbar && cur.scrollbar.ondragstop();  
};  
  
var App = new FastXDM.Client({  
  onInit: function() {  
    setTimeout(function () {  
      Community.resizeMidget();  
    }, 500);  
  },  
  authorised: function (args) {  
    var href = location.href;  
    if (href.indexOf('fieldtext=') != -1) href = href.replace('fieldtext=', 'fieldtext=');  
    else href = href + 'fieldtext=' + encodeURIComponent('');  
  
    if (href.indexOf('autologin=') != -1) href = href.replace('autologin=', 'autologin=');  
    location.href = href;  
    return;  
  },  
  unauthorised: function (args) {  
    var href = location.href;  
    if (href.indexOf('autologin=') != -1) href = href.replace('autologin=', 'autologin=');  
    href = href + 'autologin=';
```



Hands-On Demo | Function to calculate square

Pop Quiz

What makes a function impure?

A

**Same input gives same
output**

B

Depends on external variables

Pop Quiz

What makes a function impure?

A

**Same input gives same
output**

B

Depends on external variables



Math and DateTime

Math in JavaScript

- JavaScript provides built-in Math object
- Used for calculations and number operations
- No need to create custom math functions
- Helps in real-world logic



Common Math Methods

- `Math.random()` : random number (0–1)
- `Math.floor()` : round down
- `Math.ceil()` : round up
- `Math.round()` : nearest number



Generate Random Number (1-100)

```
function getRandomNumber() {  
    return  
    Math.floor(Math.random() *  
    100) + 1;  
}
```

- Math.random() gives 0-1
- Multiply by 100
- Math.floor() removes decimal
- +1 shifts range to 1-100





Hands-On Demo | Function to calculate square

Pop Quiz

Which method generates a random number?

A

Math.floor()

B

Math.random()

Pop Quiz

Which method generates a random number?

A

Math.floor()

B

Math.random()



DateTime in Javascript

DateTime in JavaScript

- JavaScript provides Date object
- Used to work with date and time
- Get current date and time
- Useful for real-world applications



Useful Date Methods

- `getFullYear()` : year
- `getMonth()` : month
(0–11)
- `getDate()` : day
- `getHours()` : hour





Hands-On Demo | Function to calculate age

Pop Quiz

Which object is used to work with date and time in JavaScript?

A

Time

B

Date

Pop Quiz

Which object is used to work with date and time in JavaScript?

A

Time

B

Date



Class Project

Integrated Practice on Functions + Math + Date

Summary

- Arrow functions provide shorter function syntax
- Arrow functions are widely used in modern JavaScript
- Scope controls variable accessibility (global and local)
- Local variables are accessible only inside functions
- Pure functions give same output for same input
- Impure functions depend on external variables
- Recursion is when a function calls itself
- Base case stops recursion
- Recursive call repeats function until base case
- Math object provides built-in calculation methods

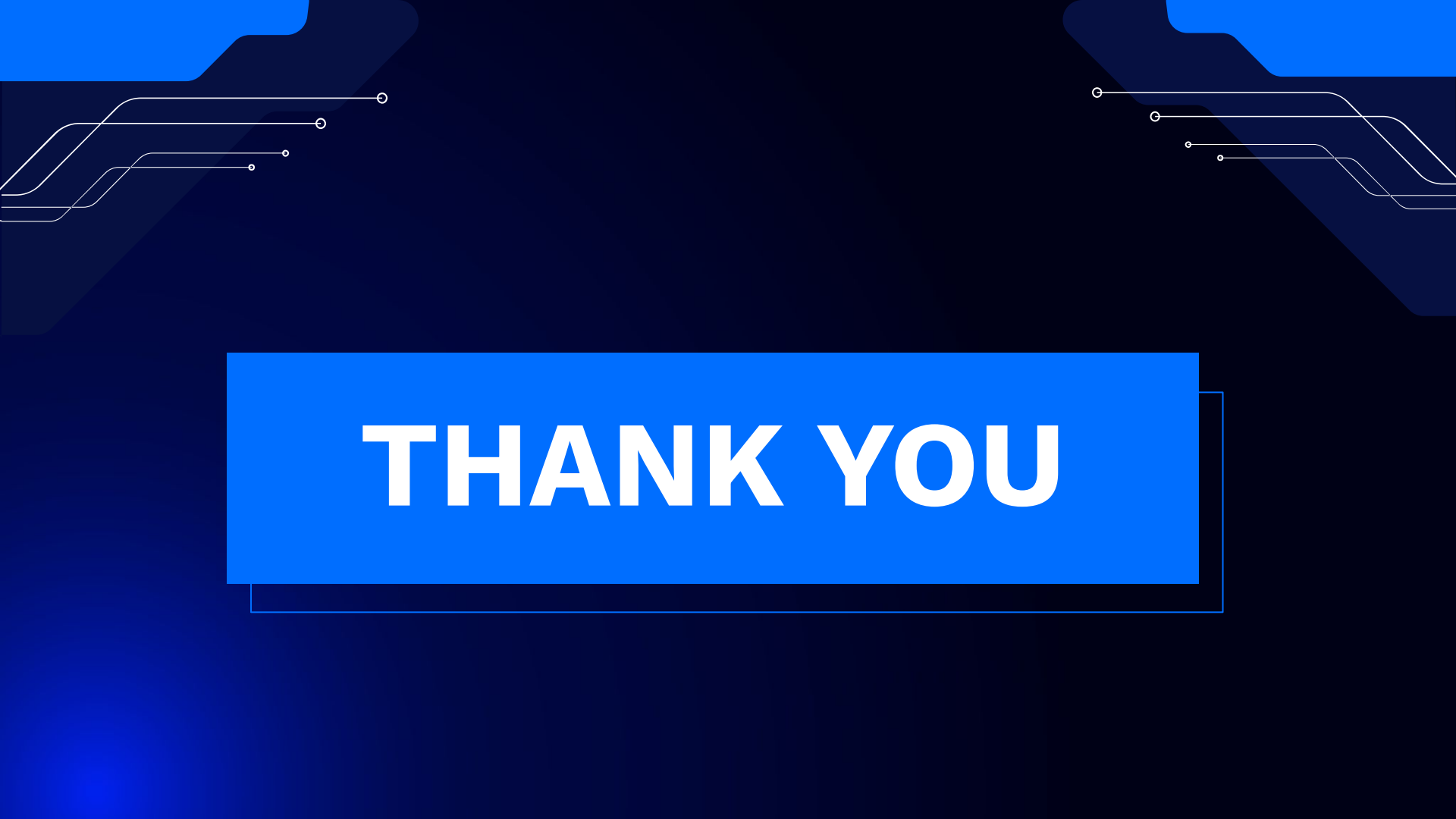




DOUBT RESOLUTION

Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.



THANK YOU